CoolDesk

AC Service Management Portal

User Flows Document · v1.0

Derived from: Architecture Document v5.2 + PostgreSQL Schema v2.0

Audience	Implementation team — engineers, QA, product
Scope	All 19 modules across 4 user roles
State machine	Forward-only lifecycle; rollback by exception only
Version	1.0 — initial release

0. Document Legend

All flows use the following notation consistently:

(START) / (END)	Flow entry or exit point with trigger or final state
[Actor: ACTION]	A human user performing an explicit UI action
[SYSTEM]	Backend-initiated operation (DB write, job, notification)
[DB]	Direct database operation called out explicitly
<DECISION>	A conditional branch point
→	Direction of flow
Indentation	Denotes a nested branch under a decision
⚠ **Edge Case**	Side-branch handling validation errors, failures, retries

State Constraint: No flow may allow an invalid job_status for its type. The validate_job_status trigger enforces this at DB level as a second layer behind the service layer.

1. Roles & Capability Matrix

CoolDesk has four distinct actor types. All authentication uses JWT tokens with role-based guards enforced on every API request.

Capability	Owner	Office Staff	Technician	Dealer
Create jobs	✓	✓	—	✓ (own)
Assign technicians	✓	✓	—	—
Advance job status	✓	✓	✓ (assigned)	—
One-state rollback	✓	✓ (pre-complete)	—	—
Full status override	✓	—	—	—
Cancel job directly	✓	✓	—	Initial state only
Submit cancel request	—	—	—	✓
Approve cancel request	✓	✓	—	—
Record payment	✓	✓	✓	—
Edit payment method	✓	✓	—	—
Edit payment amount	✓	—	—	—
Void/retain payment	✓	—	—	—
Reopen cancelled job	✓	—	—	—
Manage brands/methods	✓	—	—	—
View own jobs	✓	✓	✓ (assigned)	✓ (own)
View analytics	✓	✓	—	—

2. Job Status Reference

2.1 Installation Status States

Code	Label	Meaning
`pending_schedule`	**Pending Schedule**	Dealer-submitted only. Awaiting date assignment by office staff.
`scheduled`	**Scheduled**	Date and time confirmed. Direct jobs may start here if scheduled at creation.
`assigned`	**Assigned**	Technician assigned. In-app notification sent to technician.
`acknowledged`	**Acknowledged**	Technician confirmed receipt. Timestamp recorded.
`in_transit`	**In Transit**	Technician is en route to site.
`in_process`	**In Process**	Technician on site performing installation.
`completed`	**Completed**	Installation done. Payment recorded atomically.
`cancelled`	**Cancelled**	Terminal. Reason mandatory. Excluded from completion rate.

2.2 Complaint Status States

Code	Label	Meaning
`new`	**New**	Complaint logged. Awaiting technician assignment.
`assigned`	**Assigned**	Technician assigned. Notification sent.
`acknowledged`	**Acknowledged**	Technician confirmed receipt.
`in_transit`	**In Transit**	Technician en route.
`in_process`	**In Process**	Technician on site working on issue.
`resolved`	**Resolved**	Issue fixed. Payment recorded. Terminal.
`needs_revisit`	**Needs Revisit**	Issue unresolved. Revisit record created. Amber alert raised.
`revisit_scheduled`	**Revisit Scheduled**	Return visit date set. Amber alert cleared.
`resolved_on_revisit`	**Resolved on Revisit**	Issue fixed on a return visit. Payment recorded. Terminal.
`cancellation_requested`	**Cancellation Requested**	Dealer requested cancellation. Job frozen. Awaiting decision.
`cancelled`	**Cancelled**	Terminal. Reason mandatory.

3. Flow 1 — Customer Identity Resolution & Job Creation

Actors	Office Staff, System, (Dealer for dealer-submitted jobs)
Trigger	User opens 'Log New Job' form (office staff) OR dealer opens 'Submit Job' in dealer portal
Outputs	New job row, new or linked virtual_customers row, VCID assigned to job

3.1 Happy Path — Direct Job (Office Staff)

(START) Office staff opens 'Log New Job' form
→ **[Office Staff]** Selects job type (Installation / Complaint) and source = Direct
→ **[Office Staff]** Enters: customer name, primary phone number, address, brand, issue details (complaint) or unit details (installation)
→ **[SYSTEM]** Phone number lookup: queries customer_phones WHERE phone = entered_number
→ **<DECISION>** Phone match found?
  → **YES (Phone Match):**
→ **[SYSTEM]** Retrieves linked virtual_customers record. Displays suggestion: 'Link to [Matched Name] — [Matched Address]?'
→ **[Office Staff]** Reviews displayed name + address
→ **<DECISION>** Staff confirms link?
  → **YES:**
→ **[DB]** Uses existing vcid. Pre-fills customer_name and address fields (editable). Linked.
  → **NO (Dismiss):**
→ **[DB]** Creates new virtual_customers row. Generates new vcid for this job.
→ **[SYSTEM]** Logs timeline event: customer_link_dismissed — matched name, actor, timestamp.
  → **NO (No Phone Match):**
→ **[SYSTEM]** Secondary check: partial name + address match (Levenshtein on name, word-overlap on address)
→ **<DECISION>** Partial match found?
  → **YES:**
→ **[SYSTEM]** Displays secondary linking suggestion with matched name + address
→ **[Office Staff]** Confirms or dismisses (same branch logic as above)
  → **NO:**
→ **[SYSTEM]** No suggestion shown. Proceeds with new VCID generation.

→ **[SYSTEM]** Checks job history for linked VCID: repeat and frequent complaint detection
→ **[SYSTEM]** Sets jobs.is_repeat, is_frequent flags if thresholds met. Records window values used.
→ **<DECISION>** Is source = via_dealer? (must have dealer_id set)
  → **YES: DB enforces chk_dealer_source constraint**
  → **NO: Constraint satisfied trivially**
→ **<DECISION>** Is scheduling date provided at creation? (installation only)
  → **YES: job.status = 'scheduled'**
  → **NO (installation, dealer): job.status = 'pending_schedule'**
  → **NO (complaint): job.status = 'new'**
→ **[DB]** INSERT jobs row with vcid, status, type, source, brand_id, PII fields, version=0
→ **[DB]** INSERT job_units rows for each AC unit entered
→ **[SYSTEM]** INSERT job_timeline: status_transition event, actor_user_id = staff id
→ **[SYSTEM]** INSERT analytics update queued for business_daily and brand_daily
(END) Job created. VCID assigned. Status set per type and scheduling state.

3.2 Edge Cases — Job Creation

⚠ Brand is inactive — system blocks submission. Staff must select an active brand.

⚠️ Dealer source selected but dealer_id absent — DB constraint chk_dealer_source rejects INSERT.

⚠️ Complaint fields present on installation job (or vice versa) — service layer rejects before DB write.

⚠️ Phone number already linked to two different VCIDs — system surfaces both matches; staff selects correct one manually.

⚠️ Dismissed link is later identified as incorrect — owner or staff can manually link via customer lookup flow and update VCID.

4. Flow 2 — Dealer Job Submission

Actors	Dealer, System, Office Staff (notification recipient)
Trigger	Dealer logs in to dealer portal and opens 'Submit Job'
Outputs	New job at status new (complaint) or pending_schedule (installation). Owner + office staff notified.

4.1 Happy Path

(START) Dealer authenticated via dealer_credentials. Opens 'Submit Job' form.
→ **[Dealer]** Selects job type: Complaint or Installation
→ **[Dealer]** Enters: customer name, phone, address, brand (from dealer's linked brands only), model, unit details / issue description
→ **[SYSTEM]** Phone number lookup against customer_phones (scoped: dealer sees only New/Existing indicator, not internal VCID details)
→ **[SYSTEM]** Flags for office staff VCID review if new phone on known customer pattern
→ **<DECISION>** Dealer's brand linked in dealer_brands?
  → **NO: Brand not shown in dropdown. Dealer cannot submit.**
  → **YES: Continues**
→ **[DB]** INSERT jobs: source='via_dealer', dealer_id set, status='new' (complaint) or 'pending_schedule' (installation), version=0
→ **[DB]** INSERT job_units for unit details
→ **[SYSTEM]** INSERT notifications: dealer_job_submitted → recipient: owner + all office_staff
→ **[SYSTEM]** INSERT job_timeline: status_transition event, actor_dealer_id = dealer id
→ **[SYSTEM]** Queue analytics update: analytics_dealer_daily.jobs_submitted++
(END) Job visible in dealer's read-only history. Owner + office staff see notification.

4.2 Pending Schedule Queue (Installation)

Dealer-submitted installations appear in a dedicated 'Pending Schedule' queue in the office staff portal. They display as 'Awaiting Scheduling' in the dealer's history view.
→ **[Office Staff]** Opens Pending Schedule queue. Selects job.
→ **[Office Staff]** Sets date and time (scheduled_at). Optionally assigns technician in same action.
→ **<DECISION>** Technician assigned at same step?
  → **YES:**
→ **[DB]** UPDATE jobs: status = 'scheduled', then status = 'assigned'. Two timeline events written.
→ **[DB]** INSERT job_assignments: is_active=TRUE, assigned_by=staff_id
→ **[SYSTEM]** INSERT notification: job_assigned → recipient: technician
  → **NO (schedule only):**
→ **[DB]** UPDATE jobs: status = 'scheduled'. One timeline event.
→ **[Office Staff]** Assigns technician separately via Flow 5.
(END) Installation moves from pending_schedule → scheduled (→ assigned if combined).

4.3 Edge Cases — Dealer Submission

⚠ Dealer account is_active = FALSE — login blocked at dealer_credentials auth step.
⚠ Dealer submits brand not in dealer_brands — brand absent from dropdown; hard block.
⚠ Network failure during submission — dealer sees error. No partial INSERT committed. Dealer retries.

5. Flow 3 — Installation Job Lifecycle

Actors	Office Staff / Owner (scheduling, assignment), Technician (field), System
Trigger	Job exists at status 'scheduled' with a technician assigned
Terminal	completed or cancelled

State Gate: A technician assignment is mandatory before the job can advance beyond 'scheduled'. The service layer enforces this — no transition to 'acknowledged' is permitted without an active job_assignments row.

5.1 Happy Path

(START) Job at status: assigned. Active job_assignments row exists.
→ **[Technician]** Opens assigned job on mobile view. Taps 'Acknowledge'
→ **[DB]** UPDATE jobs: status = 'acknowledged', version++
→ **[DB]** UPDATE job_assignments: acknowledged_at = NOW()
→ **[SYSTEM]** INSERT job_timeline: status_transition. INSERT analytics: avg_acknowledgment_mins update queued
→ **[SYSTEM]** 60-second undo countdown starts on device immediately

→ **[Technician]** Taps 'In Transit' when departing for site
→ **[DB]** UPDATE jobs: status = 'in_transit', version++
→ **[SYSTEM]** Timeline event written. Undo window resets to 60 seconds.

→ **[Technician]** Arrives on site. Taps 'In Process'. Actual arrival time logged.
→ **[DB]** UPDATE jobs: status = 'in_process', actual_arrival = NOW(), version++
→ **[SYSTEM]** Derives visit_outcome: compares actual_arrival vs scheduled_at using punctuality_grace_period_mins from system_config
→ **[DB]** SET visit_outcome = on_time | late | no_show. If late: late_by_minutes calculated and stored.
→ **[SYSTEM]** Timeline event written.

→ **[Technician]** Completes installation. Taps 'Complete'. Enters payment amount and method.
→ **[SYSTEM]** Queues status transition + payment as single atomic entry in browser sync queue
→ **[SYSTEM]** Undo countdown starts on device immediately (60s from queue submission, not server confirmation)
→ **[DB]** BEGIN TRANSACTION: UPDATE jobs status='completed' version++; INSERT payments (status='collected' or 'pending'); INSERT two timeline events (status_transition + payment_recorded). COMMIT.
→ **[SYSTEM]** Conflict guard: UNIQUE on payments.job_id prevents double-payment on retry
(END) SUCCESS: Installation completed. Payment recorded. Status = completed.

5.2 Undo Window — Technician Revert

(START) Technician taps Undo within 60 seconds of any status transition
→ **<DECISION>** Has the queued entry synced to server yet?
→ **NO (still queued on device):**
→ **[SYSTEM]** Device removes both the transition AND any paired undo from the queue. Server never sees either event.
→ **YES (sync landed on server):**
→ **[SYSTEM]** Server applies rollback. If reverting Completed/Resolved: payment INSERT is also rolled back in same transaction.
→ **[DB]** UPDATE jobs: status reverts to in_process, version++. DELETE payment row if completion was undone.

→ **[SYSTEM]** Timeline event: status_undo written.
→ **[SYSTEM]** If Needs Revisit was undone: DELETE revisit row. Clear amber alert.
(END) Job returns to in_process. All paired records rolled back.

5.3 Offline Sync Handling

> **Offline Rule:** The technician PWA queues status transitions atomically. If the device is offline >60s before sync lands, the server accepts the transition but logs a 'Delayed Sync' system_event in job_timeline with occurred_at from the client timestamp.

5.4 Edge Cases — Installation Lifecycle

⚠ Optimistic lock conflict (version mismatch): UPDATE returns 0 rows → service raises HTTP 409. Client fetches latest version and re-presents to user.
⚠ No-Show: Technician never logs 'In Process' within window → system background job sets visit_outcome='no_show'. Owner dashboard flagged. Notification: no_show_flagged → owner + office staff.
⚠ Unacknowledged job: Background job monitors acknowledgment time. If threshold exceeded → notification: job_unacknowledged → owner + office staff.
⚠ Scheduling conflict at assignment time: See Flow 5 (Assignment Flow).

6. Flow 4 — Complaint Job Lifecycle

Actors	Office Staff / Owner (assignment), Technician (field), System
Trigger	Complaint job exists at status 'new'
Terminal	resolved, resolved_on_revisit, or cancelled

Key Difference from Installations: Complaints have no 'scheduled' state. Assignment IS the scheduling event. The flow is: new → assigned → acknowledged → in_transit → in_process → resolved (or needs_revisit loop).

6.1 Happy Path — Direct Resolution

(START) Complaint job at status: new.
→ **[Office Staff]** Assigns technician (see Flow 5). Job moves to 'assigned'.
→ **[SYSTEM]** Notification: job_assigned → technician
→ **[Technician]** Acknowledges, goes In Transit, arrives In Process (same as Flow 3, steps 1–3)
→ **[Technician]** Issue resolved on first visit. Taps 'Resolved'. Enters payment.
→ **[DB]** BEGIN TRANSACTION: UPDATE jobs status='resolved' version++; INSERT payments; INSERT timeline events. COMMIT.
→ **[SYSTEM]** Analytics: complaints_resolved++, first_visit_resolved++ (technician_daily and business_daily)
(END) SUCCESS: Complaint resolved. Payment recorded. Status = resolved.

6.2 Needs Revisit Branch

(START) Technician at status in_process. Issue cannot be resolved on this visit.
→ **[Technician]** Taps 'Needs Revisit'. Selects mandatory reason: part_unavailable | customer_not_home | issue_recurring | further_diagnosis_required | custom
→ **<DECISION>** Reason = custom?
  → **YES: custom_reason text field required (chk_custom_reason enforced at DB)**
  → **NO: custom_reason NULL**
→ **[DB]** UPDATE jobs: status = 'needs_revisit', version++
→ **[DB]** INSERT revisits: job_id, sequence_number = (SELECT COUNT(*)+1 FROM revisits WHERE job_id), reason, created_at
→ **[SYSTEM]** Trigger trg_chronic fires: if sequence_number >= 3 → UPDATE jobs.is_chronic = TRUE
→ **<DECISION>** is_chronic just became TRUE?
  → **YES:**
→ **[SYSTEM]** INSERT notification: third_revisit_reached → owner + office staff
→ **[SYSTEM]** Owner dashboard: chronic indicator added to job card
→ **[SYSTEM]** INSERT job_timeline: revisit_created event
→ **[SYSTEM]** Amber alert card raised on office staff portal and owner dashboard
→ **[SYSTEM]** Undo window starts (60s). If undone: DELETE revisit row, clear amber alert, revert to in_process.
(END) Job at needs_revisit. Amber alert active. Office staff must schedule revisit.

6.3 Revisit Scheduling

(START) Job at needs_revisit. Office staff opens job.
→ **[Office Staff]** Sets revisit date/time (revisits.scheduled_at). Assigns technician (inherits parent by default; can change).
→ **[DB]** UPDATE jobs: status = 'revisit_scheduled', version++
→ **[DB]** UPDATE revisits: scheduled_at = chosen datetime
→ **[DB]** INSERT revisit_assignments: is_active=TRUE
→ **[SYSTEM]** Amber alert automatically cleared

→ **[SYSTEM]** Timeline: revisit_scheduled event
→ **[SYSTEM]** Notification: job_assigned → assigned technician (revisit)
(END) Revisit scheduled. Status = revisit_scheduled. Amber cleared.

6.4 Revisit Execution

(START) Technician arrives for revisit. Job at revisit_scheduled.
→ **[Technician]** Acknowledges revisit, goes In Transit, arrives In Process (same pattern as initial visit)
→ **[DB]** UPDATE revisits: actual_arrival, visit_outcome derived from grace period logic
→ **<DECISION>** Outcome of this revisit?
→ **RESOLVED:**
→ **[Technician]** Taps 'Resolved on Revisit'. Enters payment.
→ **[DB]** UPDATE jobs: status='resolved_on_revisit', version++. INSERT payments. INSERT timeline events.
→ **[DB]** UPDATE revisits: outcome='resolved'
(END) SUCCESS: Complaint closed on revisit. Status = resolved_on_revisit.
→ **NEEDS ANOTHER REVISIT:**
→ **[Technician]** Taps 'Needs Revisit' again with new reason.
→ **[DB]** INSERT revisits: sequence_number incremented. trg_chronic fires again.
→ **[DB]** UPDATE revisits (current): outcome = 'needs_revisit'
→ **[SYSTEM]** Loop returns to 6.3 — office staff must schedule next revisit.

6.5 Repeat & Frequent Complaint Detection

→ **[SYSTEM]** On every new complaint INSERT: queries jobs WHERE vcid = new_vcid AND type='complaint' AND created_at > NOW() - repeat_window_days (from system_config)
→ **<DECISION>** Complaint count in window >= 1 prior?
→ **YES: jobs.is_repeat = TRUE. repeat_window_days_used recorded.**
→ **[SYSTEM]** INSERT notification: repeat_complaint_detected → owner + office staff
→ **[SYSTEM]** Checks frequent threshold: complaints in frequent_complaint_window_days >= frequent_complaint_threshold?
→ **<DECISION>** Frequent threshold met?
→ **YES: UPDATE virtual_customers.is_frequent = TRUE. jobs.is_frequent = TRUE. frequent fields recorded.**
→ **[SYSTEM]** INSERT notification: frequent_complaint_detected → owner + office staff

6.6 Edge Cases — Complaint Lifecycle

⚠ Technician marks resolved but payment cannot be processed offline — payment is queued with status transition as atomic entry and synced together.
⚠ Job in cancellation_requested: all technician action buttons disabled. Flow frozen until office staff decision.

7. Flow 5 — Technician Assignment & Scheduling Conflict

Actors	Office Staff or Owner (assigner), System, Technician (notified)
Trigger	Staff opens a job at status 'scheduled' (installation) or 'new' (complaint) and initiates assignment
Outputs	job_assignments row inserted. Job advances to 'assigned'. Technician notified.

7.1 Happy Path — First Assignment

(START) Staff opens unassigned job. Selects 'Assign Technician'.
→ **[Office Staff]** Selects technician from active technician list
→ **[SYSTEM]** Scheduling conflict check: queries jobs WHERE assigned technician has active assignment with scheduled_at BETWEEN [new_scheduled_at] AND [new_scheduled_at + standard_job_duration_mins from system_config]
→ **<DECISION>** Conflict detected?
  → **YES:**
→ **[SYSTEM]** Displays conflict warning: 'Technician already has job [conflicting_job_id] scheduled in this window'
→ **<DECISION>** Staff acknowledges and proceeds?
    → **YES:**
→ **[DB]** INSERT scheduling_conflicts: job_id, conflicting_job_id, technician_id, acknowledged_by, acknowledged_at
→ **[SYSTEM]** INSERT job_timeline: scheduling_conflict_ack event
    → **NO (staff selects different technician):**
→ **[SYSTEM]** Returns to technician selection. No conflict logged.
  → **NO Conflict:**
→ **[DB]** INSERT job_assignments: job_id, technician_id, is_active=TRUE, assigned_by, assigned_at
→ **[DB]** UPDATE jobs: status='assigned', version++
→ **[SYSTEM]** INSERT notification: job_assigned → technician
→ **[SYSTEM]** INSERT job_timeline: assignment event
→ **[SYSTEM]** UPDATE analytics: technician_daily.jobs_assigned++
(END) Job at status 'assigned'. Technician notified. Assignment history preserved.

7.2 Reassignment

(START) Job already has active assignment. Staff selects 'Reassign'.
→ **[Office Staff]** Selects new technician
→ **[SYSTEM]** Conflict check runs for new technician (same as above)
→ **[DB]** UPDATE job_assignments: is_active=FALSE, unassigned_at=NOW() (for current active row)
→ **[DB]** INSERT job_assignments: new row with new technician, is_active=TRUE
→ **[SYSTEM]** Partial unique index idx_job_assignments_active enforces only one active row per job
→ **[DB]** UPDATE jobs: status remains 'assigned' (or current if beyond assigned), version++
→ **[SYSTEM]** INSERT notification: job_assigned → new technician
→ **[SYSTEM]** INSERT job_timeline: reassignment event with previous and new technician
(END) Reassignment complete. Full assignment history preserved in job_assignments.

7.3 Edge Cases — Assignment

⚠ Assigning inactive/deleted technician — service layer rejects. users.is_deleted=TRUE or is_active=FALSE blocked before DB write.

⚠ Conflict acknowledged and logged — double-booking is permitted with explicit acknowledgment only. No hard block.

⚠️ Reassignment while technician is in_transit — permitted. New technician notified. Previous technician loses job from their view.

8. Flow 6 — Cancellation & Withdrawal

Actors	Office Staff, Owner, Dealer, System
Trigger	User initiates cancellation at any pre-terminal state
Terminal	cancelled

8.1 Direct Cancellation — Office Staff or Owner

(START) Staff or Owner selects 'Cancel Job' on any non-terminal, non-completed job.
→ **<DECISION>** Job status is in_process?
  → **YES:**
→ **[SYSTEM]** Displays warning: 'Work has commenced on this job. Confirm cancellation?'
→ **[Office Staff]** Must explicitly confirm
  → **NO: Proceeds directly to reason entry**
→ **[Office Staff]** Enters mandatory cancellation reason
→ **[DB]** UPDATE jobs: status='cancelled', cancelled_at=NOW(), cancelled_by=actor_id, cancellation_reason=text, version++
→ **[SYSTEM]** DB constraint chk_cancel_meta enforces cancelled_by and reason are both present
→ **[SYSTEM]** INSERT job_timeline: cancellation event
→ **[SYSTEM]** Job excluded from operational queues. Visible in history. Soft-delete NOT applied (cancellation ≠ deletion).
(END) Job at status: cancelled. Fully preserved in history.

8.2 Dealer Direct Withdrawal (Initial State Only)

> **Rule:** Dealer may withdraw directly only while job is in its initial state: 'new' for complaints, 'pending_schedule' for installations. Any state beyond this requires a cancellation request.

(START) Dealer opens job in 'new' or 'pending_schedule' status in their portal.
→ **[Dealer]** Taps 'Withdraw Job'. Enters mandatory withdrawal reason.
→ **[SYSTEM]** Service layer validates: job.source='via_dealer' AND job.dealer_id=authenticated_dealer AND status IN ('new','pending_schedule')
→ **[DB]** UPDATE jobs: status='cancelled', cancelled_at, cancellation_reason, version++. (cancelled_by set to system/dealer context.)
→ **[SYSTEM]** INSERT job_timeline: cancellation event, actor_dealer_id
(END) Job cancelled. Dealer sees 'Withdrawn' in their history.

8.3 Dealer Cancellation Request (Beyond Initial State)

(START) Dealer opens job in any status beyond initial state in their portal.
→ **[Dealer]** Taps 'Request Cancellation'. Enters mandatory reason.
→ **[SYSTEM]** Validates: no existing job_cancellation_requests for this job (UNIQUE on job_id enforced at DB level)
→ **[DB]** INSERT job_cancellation_requests: job_id, requested_by_dealer_id, reason, status='pending'
→ **[DB]** UPDATE jobs: status='cancellation_requested', version++
→ **[SYSTEM]** INSERT notifications: cancellation_request_submitted → owner + all office_staff
→ **[SYSTEM]** INSERT job_timeline: cancellation_request event
→ **[SYSTEM]** All technician action buttons disabled. Job frozen.
(END) Job at status: cancellation_requested. Awaiting staff decision.

8.4 Staff/Owner Decision on Cancellation Request

(START) Staff or Owner opens flagged job. Views cancellation request badge.
→ **<DECISION>** Decision?
→ **APPROVE:**
→ **[DB]** UPDATE job_cancellation_requests: status='approved', decided_by, decided_at. chk_decision_meta enforced.
→ **[DB]** UPDATE jobs: status='cancelled', cancellation_reason = dealer's reason, cancelled_at, cancelled_by, version++
→ **[SYSTEM]** INSERT notification: cancellation_request_outcome → dealer
→ **[SYSTEM]** INSERT job_timeline: cancellation_approved event
→ **REJECT:**
→ **[DB]** UPDATE job_cancellation_requests: status='rejected', decided_by, decided_at.
→ **[DB]** UPDATE jobs: status = status held at time of request (not advanced further), version++

> **Edge Case:** If the job's lifecycle has advanced while the request was pending, the system auto-rejects the request (status jump would be invalid). The request is marked rejected with system_event in timeline.

→ **[SYSTEM]** INSERT notification: cancellation_request_outcome → dealer
→ **[SYSTEM]** INSERT job_timeline: cancellation_rejected event
→ **[SYSTEM]** Technician action buttons re-enabled. Job continues normal lifecycle.
Dealer sees outcome (Approved or Rejected) in read-only history. Cannot submit another request for this job (DB UNIQUE constraint prevents re-insertion).
(END) Cancellation request resolved. Job either cancelled or returned to active lifecycle.

8.5 Owner Reopen Cancelled Job

(START) Owner opens a cancelled job. Taps 'Reopen'.
→ **[Owner]** Enters mandatory reopen reason
→ **[SYSTEM]** Queries job_timeline: finds last non-cancelled status (status_transition events before cancellation)
→ **[DB]** UPDATE jobs: status = last_pre_cancellation_status, cancelled_at=NULL, cancelled_by=NULL, cancellation_reason=NULL, version++
→ **[SYSTEM]** INSERT job_timeline: reopened event with reason, actor, timestamp
(END) Job returned to pre-cancellation status. Fully active in operational queues.

8.6 Edge Cases — Cancellation

⚠ Cancellation while technician has unsent queue entries (offline) — server rejects any queued transitions from technician once job.status='cancellation_requested'. Returns error on sync.
⚠ Dealer attempts second cancellation request after rejection — UNIQUE constraint on job_cancellation_requests.job_id blocks INSERT at DB level.

9. Flow 7 — Payment Recording & Management

Actors	Technician (initial record), Office Staff (method edit), Owner (amount edit, void/retain)
Trigger	Technician marks job Completed or Resolved
Outputs	payments row created 1:1 with job. Timeline events logged.

9.1 Initial Payment Recording

Atomic Operation: Status transition (completed/resolved) and payment INSERT are a single atomic transaction (see Schema Section E.2). Both are queued as one sync entry on the technician device.

(START) Technician taps Complete/Resolved. Payment form presented.
→ **[Technician]** Enters amount and selects payment method from owner-managed dropdown
→ **<DECISION>** Payment collected now or pending?
  → **Collected: payments.status = 'collected'**
  → **Pending (e.g. cheque, transfer not confirmed): payments.status = 'pending'**
→ **[DB]** BEGIN TRANSACTION: UPDATE jobs status version++; INSERT payments (job_id UNIQUE prevents duplicate on retry); INSERT timeline events x2. COMMIT.
(END) Payment recorded. Job closed (completed or resolved).

9.2 Office Staff: Correct Payment Method

(START) Office staff identifies method was entered incorrectly.
→ **<DECISION>** Job status is completed or resolved (closed)?
  → **YES: Record locked. Only Owner can edit.**
  → **NO (job still active, payment recorded): Proceeds**
→ **[Office Staff]** Selects new payment method from dropdown
→ **[DB]** UPDATE payments: payment_method_id = new_id, last_edited_by = staff_id, last_edited_at = NOW(), version++
→ **[SYSTEM]** INSERT job_timeline: payment_edited — previous value, new value, actor, timestamp
(END) Payment method corrected. Audit trail written.

9.3 Owner: Edit Payment Amount

(START) Owner identifies payment amount was entered incorrectly.
→ **[Owner]** Enters corrected amount
→ **[DB]** UPDATE payments: amount = new_amount, last_edited_by = owner_id, last_edited_at = NOW(), version++ (optimistic lock applied)
→ **[SYSTEM]** INSERT job_timeline: payment_edited — previous amount, new amount, actor, timestamp
(END) Amount corrected. Timeline audit entry written.

9.4 Owner: Void or Retain Payment After Status Reversal

Trigger Condition: Owner reverses a 'completed' or 'resolved' status OUTSIDE the 60-second undo window AND a payments record exists for the job.

(START) Owner initiates status reversal on closed job.
→ **[SYSTEM]** Detects: payments row exists for this job_id. Presents mandatory decision prompt — cannot be bypassed.
→ **<DECISION>** Owner's decision?
  → **RETAIN:**
→ **[DB]** UPDATE payments: retained_note = 'Recorded on prior completion'. No amount or status change.

→ **[SYSTEM]** INSERT job_timeline: payment_retained — actor, timestamp
→ **VOID:**
→ **[Owner]** Enters mandatory void reason
→ **[DB]** UPDATE payments: voided_at=NOW(), voided_by=owner_id, void_reason=text. chk_void_meta enforced.
→ **[SYSTEM]** INSERT job_timeline: payment_void — previous values, reason, actor, timestamp. Also: rollback_payment_voided event.
→ **[DB]** UPDATE jobs: status = 'in_process', version++ (after decision made)
→ **[SYSTEM]** INSERT job_timeline: status rollback event (owner override)
(END) Payment decision recorded. Job returned to in_process for re-completion.

9.5 Edge Cases — Payment

⚠ Technician tries to edit payment after recording — UI prevents it. Cannot modify once submitted.

⚠ Optimistic lock conflict on payment update — UPDATE returns 0 rows → HTTP 409. Actor re-fetches and retries.

⚠ Payment method deactivated after job completed — historical record retains method_id with RESTRICT FK. Method remains in data but not offered in new dropdowns.

10. Flow 8 — Office Staff One-State Rollback

Actors	Office Staff, System
Trigger	Staff identifies a technician made an incorrect status transition
Constraint	Pre-completion and pre-resolution states only. Completed and Resolved are owner-only reversals.

10.1 Happy Path

(START) Staff opens job. Current status is in_transit, in_process, acknowledged, or assigned.
→ **<DECISION>** Is status completed, resolved, or resolved_on_revisit?
  → **YES: Rollback blocked. Only owner can reverse these. Staff is shown message.**
  → **NO: Rollback available**
→ **<DECISION>** Is status cancelled?
  → **YES: Rollback blocked. Only owner can reopen cancelled jobs.**
  → **NO: Rollback available**
→ **[Office Staff]** Taps 'Roll Back Status'. Enters mandatory reason.
→ **[SYSTEM]** Determines target: previous status (one step back in the valid state sequence for this job type)
→ **[DB]** UPDATE jobs: status = previous_status, version++
→ **[SYSTEM]** INSERT job_timeline: status_rollback event — previous status, reverted status, reason, actor, timestamp
(END) Job rolled back one state. Reason permanently logged.

10.2 Rollback State Map

in_transit →	acknowledged
in_process →	in_transit
acknowledged →	assigned
revisit_scheduled →	needs_revisit (amber alert re-raised)
assigned →	new (complaint) or scheduled (installation)

10.3 Edge Cases — Rollback

⚠ Rolling back from needs_revisit: revisit record is NOT deleted by rollback (unlike undo window). Revisit remains as a record. Amber alert remains active. Discuss with product owner if record deletion is required in this case.

⚠ Rolling back assigned while technician is actively working on a different status (race condition) — optimistic lock version guard prevents stale write.

11. Flow 9 — Visit Outcome & Punctuality Tracking

Actors	System (automated), Background Job
Trigger	Technician logs actual arrival (taps 'In Process') OR visit window expires with no arrival
Applies to	All visit types: initial installation, initial complaint, revisit

11.1 On-Time / Late Derivation

(START) Technician taps 'In Process'. actual_arrival recorded.
→ **[SYSTEM]** Reads scheduled_at from jobs (or revisits for revisit visits) and punctuality_grace_period_mins from system_config
→ **<DECISION>** actual_arrival <= scheduled_at + grace_period?
  → **YES: visit_outcome = 'on_time'**
  → **NO:**
→ **[DB]** visit_outcome = 'late'. late_by_minutes = EXTRACT(EPOCH FROM actual_arrival - scheduled_at)/60 (rounded). CHECK late_by_minutes >= 0.
→ **[SYSTEM]** Analytics update: on_time_count or late_count incremented for technician_daily

11.2 No-Show Detection

(START) Background job runs periodically. Checks jobs/revisits WHERE scheduled_at has passed AND actual_arrival IS NULL AND status != 'cancelled'.
→ **[SYSTEM]** visit_outcome = 'no_show' set on job/revisit row
→ **[SYSTEM]** INSERT notification: no_show_flagged → owner + office staff
→ **[SYSTEM]** Owner dashboard: job flagged for follow-up
(END) No-show logged. Owner alerted.

11.3 Rescheduled Outcome

→ **[Office Staff]** Modifies scheduled_at before visit occurs
→ **[DB]** visit_outcome = 'rescheduled'. New scheduled_at saved. Previous slot lost (no history of prior schedule maintained unless logged in timeline).
→ **[SYSTEM]** INSERT job_timeline: note of reschedule, actor, timestamp

12. Flow 10 — Customer Review (Module 19)

Actors	Technician, Customer (external), System, Owner (config)
Trigger	Job reaches completed or resolved_on_revisit status AND review_mode != 'off'
Outputs	customer_reviews row. Review link generated. 48-hour expiry window.

12.1 Happy Path — Optional Mode

(START) Job marked completed/resolved. review_mode = 'optional' in system_config.
→ **[SYSTEM]** Displays review link prompt to technician after status transition confirmed
→ **<DECISION>** Technician chooses to generate link?
  → **NO: Technician dismisses. No review record created. Job fully closed.**
  → **YES:**
→ **[DB]** INSERT customer_reviews: job_id, technician_id, brand_id, review_token (UUID), link_generated_at. expires_at generated automatically as link_generated_at + 48h.
→ **[Technician]** Copies review link. Shares with customer via WhatsApp or SMS.
→ **[Customer]** Opens link on own device. Submits star_rating (1–5) and optional review_text.
→ **<DECISION>** Rating submitted before expires_at?
  → **YES:**
→ **[DB]** UPDATE customer_reviews: submitted_at, star_rating, review_text. is_low_rated generated column auto-sets if rating <= 2.
→ **[SYSTEM]** If is_low_rated = TRUE: flag visible on owner dashboard.
→ **[SYSTEM]** Analytics: avg_star_rating updated in technician_daily
  → **NO (expired):**
→ **[SYSTEM]** 'Review Pending' indicator cleared automatically. Review record remains with submitted_at = NULL.
(END) Review cycle complete.

12.2 Mandatory Mode

Identical to Optional, except:

- Technician cannot proceed past the review prompt without generating the link.
- 'Review Pending' indicator shown on job until customer submits OR 48-hour window expires.
- Expiry clears the indicator automatically — it does not block the job from being treated as closed.

12.3 Mode Changes by Owner

Owner can switch mode (off / optional / mandatory) at any time in system settings. Changes apply only to new completions. Already-completed jobs are unaffected.

13. Flow 11 — Dealer Account Management

Actors	Owner
Trigger	Owner opens Settings → Dealer Management

13.1 Create Dealer Account

(START) Owner opens 'Add Dealer' form.
→ **[Owner]** Enters: business_name, contact_name, email, region. Assigns brands (dealer_brands).
→ **[Owner]** Sets initial password for dealer portal access.
→ **[DB]** INSERT dealers row (is_active=TRUE, is_deleted=FALSE). INSERT dealer_credentials (password_hash). INSERT dealer_brands rows.
(END) Dealer can now log in. Assigned brands available in submission form.

13.2 Deactivate / Reactivate Dealer

→ **[Owner]** Toggles is_active on dealer record
→ **[DB]** UPDATE dealers: is_active = FALSE/TRUE. (is_deleted remains FALSE — historical data preserved.)
Deactivated dealer: login blocked. Existing job history retained. dealer_brands links retained.

13.3 Brand Assignment

→ **[Owner]** Adds or removes brand links for a dealer
→ **[DB]** INSERT or DELETE dealer_brands rows. brands.id FK has ON DELETE RESTRICT — brand must be deactivated, not deleted.

14. Flow 12 — System Configuration Management

Actors	Owner exclusively
Trigger	Owner opens Settings panel
Storage	system_config key-value table

14.1 Configurable Items

Config Key	Default	Effect
`repeat_complaint_window_days`	30 days	Window for repeat complaint detection
`frequent_complaint_threshold`	3 complaints	Count threshold for frequent flag
`frequent_complaint_window_days`	90 days	Rolling window for frequent detection
`punctuality_grace_period_mins`	15 minutes	Grace period before visit is 'late'
`customer_review_mode`	off	Controls Module 19 behaviour
`undo_window_seconds`	60 seconds	Device-side undo countdown (system constant; not truly runtime configurable yet)
`standard_job_duration_mins`	120 minutes (recommended)	Used for scheduling conflict window check. Add to system_config.

14.2 Update Flow

(START) Owner selects config item. Edits value.
→ **[DB]** UPDATE system_config: value = new_value, updated_by = owner_id, updated_at = NOW()
Effect on repeat/frequent flags: thresholds apply to new evaluations only. Existing flags are point-in-time snapshots (see Schema Assumption A-2). Retroactive recalculation requires a separate background job if needed.
(END) Config updated. Applied to all new events from this point forward.

15. Flow 13 — Analytics Update Pipeline

Actors	System (NestJS analytics worker)
Trigger	Any job mutation: status change, payment, assignment, cancellation, rollback, revisit outcome
Storage	analytics_technician_daily, analytics_business_daily, analytics_brand_daily, analytics_dealer_daily

Idempotency: The analytics worker tracks processed timeline_event_ids in analytics_processed_events. On retry, it skips already-applied events. This prevents double-counting on retried job mutations (Schema Section E.3).

15.1 Update Pipeline

(START) A status_transition or payment_recorded event is inserted into job_timeline.
→ **[SYSTEM]** Analytics worker queries job_timeline for unprocessed events (NOT IN analytics_processed_events)
→ **[SYSTEM]** For each event: determines which analytics tables are affected by event_type and job fields
→ **[DB]** UPSERT into relevant analytics_*_daily tables using ON CONFLICT (entity_id, date) DO UPDATE
→ **[DB]** INSERT analytics_processed_events: timeline_event_id, processed_at. ON CONFLICT DO NOTHING (idempotent)
(END) Analytics tables reflect near-real-time state. Dashboards query precomputed tables, not jobs directly.

15.2 Analytics Scope by Role

Owner	Full access: all technician scorecards, business-wide metrics, dealer analytics, brand analytics
Office Staff	Same as owner for operational views. Scoped to their own data for personal actions.
Technician	No analytics access
Dealer	No analytics access. Read-only job history only.

16. Flow 14 — Notification Centre

Actors	System (sender), All roles (recipients)
Delivery	In-app notification centre only. No push notifications. Persistent with read/unread state.

16.1 Notification Events & Recipients

Event	Owner	Office Staff	Technician	Dealer
New dealer job submitted	✓	✓	—	—
Job assigned to technician	—	—	✓	—
Dealer cancellation request	✓	✓	—	—
Cancellation request outcome	—	—	—	✓
Job unacknowledged (threshold exceeded)	✓	✓	—	—
No-show flagged	✓	✓	—	—
Repeat / frequent complaint	✓	✓	—	—
Third revisit (chronic)	✓	✓	—	—

16.2 Deduplication

Partial unique indexes on notifications (idx_notif_dedup_user and idx_notif_dedup_dealer) prevent duplicate notifications from retried background workers. ON CONFLICT DO NOTHING is used on INSERT.

16.3 Notification Read Flow

→ **[Any User]** Opens notification centre icon in nav bar. Badge count shown for unread.
→ **[Any User]** Clicks a notification → navigates directly to relevant job
→ **[DB]** UPDATE notifications: is_read=TRUE, read_at=NOW()

17. Residual Gaps & Implementation Notes

Status: All 21 pre-generation clarification questions have been answered. The items below are design decisions or schema gaps discovered during flow construction that require team confirmation before implementation.

17.1 Schema Gap — standard_job_duration_mins

The scheduling conflict check (Flow 5) requires a job duration estimate to compute the conflict time window. This key is absent from system_config. It must be added as a seed value (suggested: 120 minutes) before the conflict detection feature can be implemented.

- Action: Add standard_job_duration_mins to system_config seed in seed_system_config().

17.2 Schema Gap — analytics_processed_events

The idempotent analytics worker pattern references an analytics_processed_events table (Schema Section E.3). This table is defined inline in the schema documentation but is not listed in the Section A entity breakdown or given a section number. It must be included in the migration order.

- Action: Add analytics_processed_events to the migration sequence after job_timeline.

17.3 Rollback From needs_revisit by Office Staff

If office staff roll back from needs_revisit to in_process, the revisit record created by that transition is NOT deleted (unlike the technician undo window, which does delete it). The amber alert technically remains tied to the revisit record. Clarify with product owner: should rollback from needs_revisit also delete the revisit row, or leave it as a historical record?

17.4 Rescheduled Visit — No Schedule History

When a scheduled_at is changed (rescheduled outcome), the previous scheduled datetime is overwritten. There is no schedule history table. If auditing of prior schedule slots is needed, a job_timeline note event should be explicitly written with the old value before the UPDATE.

17.5 Dealer Visibility on VCID Conflicts

When a dealer submits a job with a phone number that matches an existing VCID from a different dealer, the system currently only flags for internal office staff review. Confirm: should the dealer receive any indication, or should this remain fully internal?

17.6 Payment Status Lifecycle

A payment created with status='pending' has no defined workflow to transition to 'collected'. Confirm: does office staff or the owner manually update payment status from pending to collected (e.g. when a cheque clears)? This flow is not defined in the architecture document and will need an explicit UI control and timeline event.

17.7 Auto-Rejection of Stale Cancellation Requests

Per the answer to Q8: if a job's status has advanced while a cancellation request is pending, the request should be auto-rejected. This requires a background job or a service-layer check on every status transition. The trigger condition and system_event timeline entry format for this auto-rejection should be specified before implementation.

End of CoolDesk User Flows Document v1.0

Derived from Architecture v5.2 and Schema v2.0. All flows are implementation-ready pending resolution of Section 17 items.